

Konstrukce haldy v lineárním čase

- výchozí rozložení dat představuje úplný binární strom hloubky d (bez uspořádání hodnot do haldy)
- nejprve postavíme „haldy“ z podstromů, jejichž kořeny mají hloubku $d-1$, potom pro $d-2$, ... atd., až do kořene celé haldy
- stavění hald se provádí záměnami hodnot od kořene k listům (výměna s menším z obou synů)

Časová složitost

hloubka	počet uzlů	max. počet výměn pro každý z nich
0	2^0	d
1	2^1	$d-1$
...	...	...
j	2^j	$d-j$
...	...	...
$d-1$	2^{d-1}	1

Pozorování: Při tomto postupu konstrukce haldy má hodně uzlů malý maximální počet výměn a jen málo z nich může absolvovat výměn hodně → celková časová složitost **$O(N)$** .

Celkem se provede výměn (viz tabulka):

$$\begin{aligned} \sum_{j=0}^{d-1} 2^j (d-j) &= d \sum_{j=0}^{d-1} 2^j - \sum_{j=0}^{d-1} j \cdot 2^j = \\ &= d \cdot (2^d - 1) - ((d-2) \cdot 2^d + 2) = O(2^d) = O(N) \end{aligned}$$

... viz dva důkazy matematickou indukcí dále

Důkaz matematickou indukcí č.1: $\sum_{j=0}^{d-1} 2^j = 2^d - 1$

1. pro $d = 1$ zjevně platí

2. necht' platí pro všechna $d < D$, dokazujeme platnost pro $D > 1$:

$$\sum_{j=0}^{D-1} 2^j = \sum_{j=0}^{D-2} 2^j + 2^{D-1} = (2^{D-1} - 1) + 2^{D-1} = 2 \cdot 2^{D-1} - 1 = 2^D - 1$$

↑

podle indukčního předpokladu

qed

Důkaz matematickou indukcí č.2: $\sum_{j=0}^{d-1} j \cdot 2^j = (d-2) \cdot 2^d + 2$

1. pro $d=1$ zjevně platí

2. necht' platí pro všechna $d < D$, dokazujeme platnost pro $D > 1$:

$$\sum_{j=0}^{D-1} j \cdot 2^j = \sum_{j=0}^{D-2} j \cdot 2^j + (D-1) \cdot 2^{D-1} = \left((D-3) \cdot 2^{D-1} + 2 \right) + (D-1) \cdot 2^{D-1} =$$

↑
podle indukčního předpokladu

$$= D \cdot 2^{D-1} - 3 \cdot 2^{D-1} + D \cdot 2^{D-1} - 2^{D-1} + 2 = 2 \cdot D \cdot 2^{D-1} - 4 \cdot 2^{D-1} + 2 =$$

$$= D \cdot 2^D - 2 \cdot 2^D + 2 = (D-2) \cdot 2^D + 2 \quad \text{qed}$$

Prioritní fronta

- podobné jako fronta, prvky se „předbíhají“ podle svých priorit
- zachování vzájemného pořadí mezi prvky téže priority požadovat můžeme, ale nemusíme (záleží na konkrétní aplikaci)

Možnosti implementace:

- seznam (pole, LSS), do něhož zařazujeme podle priority
- seznam (pole, LSS), z něhož vybíráme podle priority
- samostatné seznamy (fronty) pro každou hodnotu priority
 - pokud tyto hodnoty známe a není jich mnoho
- halda řazená podle priorit
 - pokud nepožadujeme zachovat pořadí mezi prvky téže priority
- halda řazená podle dvojic (priorita, čas příchodu)
 - pokud požadujeme zachovat vzájemné pořadí prvků téže priority

Slovník (dictionary)

- uchovává dvojice *klíč – hodnota*
- klíč je jednoznačný
- uložené údaje se vyhledávají podle klíče (indexuje se klíčem)
- „asociativní pole“

- abstraktní datový typ s operacemi
vyhledej(klíč), vlož(klíč, hodnota) a vymaž(klíč)

- některé programovací jazyky přímo obsahují implementaci slovníku
- Python:
 - standardní třída `dict` (dictionary)
 - v knihovně `collections` podtřídy `defaultdict`, `Counter`

- klíč může mít jakoukoliv neměnitelnou hodnotu,
dokonce třeba každý záznam má klíč jiného typu

Implementace slovníku – pomocí hešování

(alternativně lze slovník implementovat pomocí vyhledávacích stromů)

klíč → **hešovací funkce** → index v poli (**hešovací tabulka**)

rozsah možných klíčů bývá výrazně větší než rozsah přípustných indexů

– např. klíč je rodné číslo člověka, ale pro uložení několika set lidí nám stačí pole indexované 0..999

⇒ hešovací funkce nebývá prostá ⇒ vznikají **kolize**
(hešovací funkce více záznamům přidělí stejný index v poli)

Minimalizace počtu kolizí:

- dobrá, tedy rovnoměrně rozptylující hešovací funkce
 - navrhne ji „na míru“, pokud víme, jak budou vypadat klíče
- dostatečný rozsah indexů vzhledem k počtu ukládaných záznamů
 - zaplněnost hešovací tabulky nejvýše do 90%, raději méně
 - při vyšším zaplnění je třeba přehešovat celou tabulku

Řešení kolizí:

- separátní řetězení
- otevřená adresace
(dvojitě hešování, sekundární transformace klíče)

Řešení kolizí pomocí separátního řetězení:

- v hešovací tabulce je na indexu x uložen seznam všech záznamů, jejichž klíč hešovací funkce zobrazila na index x
 - při vyhledávání záznamu podle známého klíče se na tento klíč použije hešovací funkce, tím získáme index do hešovací tabulky a na tomto indexu najdeme seznam uložených záznamů; tento seznam sekvenčně projdeme a podle známého klíče v něm vyhledáme náš záznam
- časová složitost přístupu k prvku je v nejhorším případě **lineární** vzhledem k počtu prvků uložených ve slovníku (v průměrném případě je ovšem **konstantní**)

Řešení kolizí pomocí otevřené adresace:

- všechny prvky jsou uloženy přímo v hešovací tabulce
 - při vkládání prvku se na klíč použije hešovací funkce, tím získáme index do hešovací tabulky; je-li toto místo v tabulce již obsazené, přejdeme na náhradní index
 - lineární zkoušení $(\text{index} + 1) \bmod \text{rozsah tabulky}$
 - sekundární transformační funkce $\text{index} \rightarrow \text{nový index}$
 - pozor při vypouštění prvku:
 - místo v tabulce musí zůstat zablokované!
 - uvolní se až v případě přehešování tabulky
- asymptotická časová složitost přístupu k prvku je stejná jako v předchozím případě

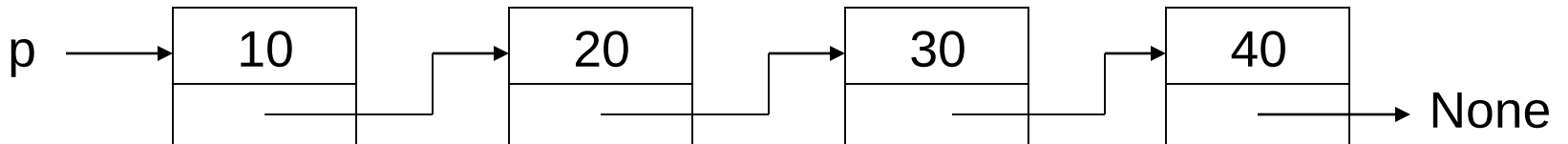
Implementace slovníku v Pythonu

- hešování s otevřenou adresací
- klíčem je řetězec bitů (bez ohledu na typ použitého klíče)
- přehešování celého slovníku, když se volný prostor sníží na 1/3

Dejte pozor při určování časové složitosti v nejhorším případě u algoritmů, které používají datovou strukturu slovník (dict v Pythonu).

Lineární spojový seznam

```
class Uzel:  
    """uzel spojového seznamu"""  
  
    def __init__(self, x = None, dal = None):  
        self.info = x                # uložená hodnota  
        self.dalsi = dal             # následník
```



```
# vytvoření spojového seznamu p s hodnotami 10 20 30
p = Uzel(10)
q = Uzel(20)
r = Uzel(30)
p.dalsi = q
q.dalsi = r

p = Uzel(10, Uzel(20, Uzel(30)))

# průchod a výpis
s = p
while s != None:
    print(s.info, end = " ")
    s = s.dalsi
print()
```

```
# poslední uzel
if p == None:
    print("prazdny seznam")
else:
    s = p
    while s.dalsi != None:
        s = s.dalsi
    print(s.info)
```

```
# vyhledání zadané hodnoty
hodnota = 20
print("hledáme", hodnota)
s = p
while s != None and s.info != hodnota:
    s = s.dalsi
if s == None:
    print("nenalezen")
else:
    print("nalezen", s.info)
```

```
# přidání na začátek seznamu  
t = Uzel(40)  
t.dalsi = p  
p = t
```

```
# přidání na konec seznamu  
if p == None:  
    p = Uzel(50)  
else:  
    s = p  
    while s.dalsi != None:  
        s = s.dalsi  
    s.dalsi = Uzel(50)
```


Operace se spojovým seznamem

- určit počet prvků
- vypsát všechny hodnoty
- nalezení posledního prvku
- vyhledání prvku s danou hodnotou
- přidání prvku na začátek, na konec seznamu
- vytvoření seznamu z dat na vstupu
- vytvoření kopie seznamu
- přidání prvku na dané místo
- přidání prvku do uspořádaného seznamu (na správné místo)
- odebrání prvku ze začátku, z konce seznamu
- odebrání daného prvku

Operace se spojovým seznamem (pokr.)

- zrušení všech prvků v seznamu s danou hodnotou
- obrácení pořadí prvků v seznamu
- uspořádání prvků v seznamu podle hodnoty
- spojení dvou seznamů do jednoho (ze sebe)
- slití dvou uspořádaných seznamů do jednoho (merge)
- rozdělení seznamu do dvou (podle pozice lichý/sudý)
- rozdělení seznamu do dvou (podle hodnoty lichý/sudý)

Příklady použití lineárních spojových seznamů

Zásobník

- realizován jednoduchým LSS, ukazatel na začátek seznamu ukazuje na vrchol zásobníku (dno zásobníku = **None**)
- prázdný zásobník = prázdný LSS
- přidávání i odebírání prvků se provádí **na začátku** LSS – snadné

```
class Zasobnik:
    """zásobník uložený v seznamu"""

    def __init__(self):
        self.s = []                # prázdný zásobník

    def pridej(self, x):
        self.s.append(x)

    def odeber(self):
        return self.s.pop()
```

```

class Zasobnik:
    """zásobník jako spojový seznam"""

    def __init__(self):
        self.p = None                                # prázdný zásobník

    def pridej(self, x):                                # na začátek
        q = Uzel(x)
        q.dalsi = self.p
        self.p = q

    def odeber(self):                                  # ze začátku
        q = self.p
        self.p = self.p.dalsi
        return q.info

```

```
z = Zasobnik()  
#je jedno, kterou implementaci třídy Zasobnik použijeme  
  
z.pridej(20)  
z.pridej(30)  
print(z.odeber())  
print(z.odeber())
```

Poznámka: v metodě odeber() jsme pro jednoduchost nekontrolovali, zda zásobník není prázdný.

Fronta

- realizována jednoduchým LSS a dvěma ukazateli:
na začátek (tj. na první prvek LSS) a
na konec (tj. na poslední prvek LSS)
- na začátku LSS se dobře přidává i odebírání prvek,
na konci LSS se dobře přidává, ale špatně odebírání
→ **na začátku LSS bude odchod, na konci LSS bude příchod**
(tzn. každý ukazuje na toho, kdo přišel po něm,
což je obráceně, než ve frontách v běžném životě)
- prázdná fronta = prázdný LSS

Jiná možná realizace: LSS s hlavou
(snáze se pak programují operace s dočasně prázdnou frontou)

Dlouhá čísla

- uložena po cifrách nebo po skupinách cifer podobně jako v poli
- nejsme omezeni maximálním možným počtem cifer v čísle
- směr řazení LSS závisí na prováděných operacích,
např. pro sčítání nebo násobení odzadu od cifer nejnižšího řádu,
pro výpis ale potřebujeme procházet cifry odpředu
→ bude nutno otáčet seznam nebo použít obousměrný seznam

Polynomy

- v každém prvku LSS uložen koeficient a exponent jednoho členu polynomu, členy s nulovým koeficientem se do LSS neukládají
- vhodné pro „řídké polynomy“ vyšších stupňů
- pro provádění operací je výhodné mít LSS uspořádaný (vzestupně nebo sestupně) podle hodnot exponentu